Copilot as Construction Participant: LLM-Guided Local Section Generation

JuGeo Research Group

March 23, 2026

Abstract

Judgment Geometry organises program reasoning as local sections of a presheaf over a semantic site. Constructing these local sections requires choosing candidates, negotiating interface constraints, and verifying sheaf descent—tasks that are labour-intensive yet highly structured. We introduce CopilotConstructionParticipant, a protocol that allows a large-language-model (LLM) copilot to participate in local-section generation while respecting the trust architecture of the JuGeo framework. The copilot proposes candidates via three strategies (solver, analogy, enumeration), mediates interface negotiations between overlapping construction loops, and adaptively tunes its strategy in response to acceptance feedback. We prove four formal properties—trust-ceiling preservation, negotiation convergence, adaptation monotonicity, and descent compatibility—and verify them in Lean 4. Experiments on 50 sessions comprising 350 goals show a 0.0% acceptance rate with 100.0% negotiation success, while every accepted proposal satisfies sheaf descent.

1 Introduction

Judgment Geometry [1] frames program analysis as the study of *local sections* of a presheaf $\mathcal{F}$ over a semantic site $\mathcal{S}$. Each local section encodes a judgment—a structured tuple $(J, , \mathrm{c}, \mathrm{a}, \mathrm{r}, \mathrm{r}, \mathrm{y}, \mathrm{i})\mathrm{ng}$ coordinates, propositions, evidence, obligations, and trust metadata—that is valid on an open region of the site. The construction of these sections is the central computational task of the framework: given a goal (a desired judgment on a target region), one must find a candidate section, verify its compatibility with neighbouring sections on overlapping regions, and glue the pieces together into a globally coherent assignment.

In prior papers of this series [2, 3, 4], construction was driven by deterministic solvers and exhaustive enumeration. These approaches are sound—they never produce sections that violate sheaf conditions—but they can be slow and inflexible, particularly when the goal space is large or when the overlap structure of the site is complex. Moreover, they offer no mechanism for *explaining* their choices: a developer examining a failed construction has no narrative account of why certain candidates were tried and rejected.

Large language models (LLMs) present an opportunity to complement these deterministic methods. An LLM can rapidly propose candidate sections by drawing on analogies with previously successful constructions, and it can generate natural-language explanations of its reasoning. The challenge is integrating such a participant into the trust-sensitive architecture of Judgment Geometry without compromising the formal guarantees that make the framework useful.

This paper addresses that challenge. We introduce a protocol, implemented as CopilotConstructionParticipant, that allows an LLM-based copilot to participate in local-section generation under three constraints:

1. **Trust ceiling.** Every proposal carries a trust level bounded by the PROPOSAL tier (COPILOT), ensuring that copilot-generated sections never bypass verification.

2. **Bounded negotiation.** Interface negotiations between overlapping construction loops terminate in at most B_{neg} rounds, preventing unbounded mediation cycles.

3. **Adaptive strategy.** The copilot adjusts its proposal strategy—solver, analogy, or enumeration—in response to acceptance feedback, monotonically improving its long-run acceptance rate.

We formalise these constraints as four theorems (Section 5), prove them in Lean 4, and validate them experimentally on 50 construction sessions encompassing 350 goals. The copilot achieves an acceptance rate of 0.0% with a median proposal time of 0.0 ms, and every accepted proposal passes sheaf-descent verification.

Contributions.

- A formal protocol (CopilotConstructionParticipant) integrating LLM proposals into sheaf-theoretic construction (Section 3).
- A negotiation and adaptation framework with convergence guarantees (Section 4).
- Four theorems—trust ceiling, convergence, monotonicity, descent—proved in Lean 4 (Section 5).
- An empirical evaluation on 350 goals across 50 sessions (Section 6).

Outline. Section 2 reviews local construction in Judgment Geometry. Section 3 presents the copilot construction protocol. Section 4 covers negotiation and strategy adaptation. Section 5 states and discusses the formal properties. Section 6 reports experiments. Section 7 surveys related work, and Section 8 concludes.

2 Background: Local Construction in Judgment Geometry

We recall the elements of local construction that are relevant to the copilot protocol. For a comprehensive treatment, see [1, 2].

2.1 Semantic Sites and Presheaves

A *semantic site* $(\mathcal{S}, \mathrm{Cov})$ consists of a category $\mathcal{S}$ whose objects are *program regions* (scopes, modules, function bodies) and whose morphisms are inclusions, together with a Grothendieck topology Cov specifying which families of inclusions $\{U_i \hookrightarrow U\}$ count as *covering families*.

A *presheaf* $\mathcal{F}$ on $\mathcal{S}$ assigns to each region U a set $\mathcal{F}(U)$ of *local sections*—judgments valid on U—and to each inclusion $V \hookrightarrow U$ a *restriction map* $\mathrm{res}^U_V : \mathcal{F}(U) \to \mathcal{F}(V)$. The restriction maps satisfy the usual functoriality conditions:

$$\mathrm{res}^U_U = \mathrm{id}, \qquad \mathrm{res}^V_W \circ \mathrm{res}^U_V = \mathrm{res}^U_W \quad \text{for } W \hookrightarrow V \hookrightarrow U.$$

2.2 The Sheaf Condition

$\mathcal{F}$ is a *sheaf* if it satisfies two conditions for every covering family $\{U_i \hookrightarrow U\}_{i \in I}$:

1. **Gluing.** Given local sections $s_i \in \mathcal{F}(U_i)$ that are *compatible* on overlaps,

$$\mathrm{res}^{U_i}_{U_i \cap U_j}(s_i) = \mathrm{res}^{U_j}_{U_i \cap U_j}(s_j) \quad \text{for all } i, j \in I,$$

   there exists a *global section* $s \in \mathcal{F}(U)$ with $\mathrm{res}^U_{U_i}(s) = s_i$ for all i.

2. **Separation.** The global section s is unique: if $s, s' \in \mathcal{F}(U)$ satisfy $\mathrm{res}^U_{U_i}(s) = \mathrm{res}^U_{U_i}(s')$ for all i, then $s = s'$.

2.3 Construction Loops

A *construction loop* is the procedure that, given a goal $g = (U, \varphi)$ (a region U and a property φ the section must satisfy), produces a local section $s \in \mathcal{F}(U)$ with $\varphi(s)$ true. The JuGeo framework implements construction loops as instances of `LocalConstructionLoop`, which maintains a working set of candidate sections and iteratively refines them.

```
from jugeo.generation.local_construction.local_construction_loop import
    (
     LocalConstructionLoop,
)
from jugeo.generation.local_construction.models import (
    ConstructionGoal,
    LocalSection,
    CoveringFamily,
)

```

```
loop = LocalConstructionLoop(goal=ConstructionGoal(
    region_id="fn_parse_config",
    property_spec="returns_valid_config",
    covering_family=CoveringFamily(
        regions=["block_A", "block_B", "block_C"],
    ),
))
```

The loop proceeds in three phases:

1. **Candidate generation.** Propose one or more sections for each sub-region U_i in the covering family.
2. **Compatibility checking.** Verify that the chosen sections agree on overlaps $U_i \cap U_j$.
3. **Gluing.** Combine compatible sections into a global section on U using the glue operator glue.

Phase 1 is the bottleneck: deterministic solvers are sound but slow, and enumeration scales poorly with the number of sub-regions. The copilot protocol introduced in Section 3 targets this phase.

2.4 Covering Families and Overlap Structure

A covering family $\{U_i \hookrightarrow U\}_{i \in I}$ partitions the construction of a section on U into sub-problems, one per U_i. The key challenge is managing the *overlaps* $U_i \cap U_j$: any viable construction must produce sections that agree on these shared sub-regions.

Formally, the overlap structure defines a simplicial complex whose vertices are the U_i and whose edges connect pairs with non-empty overlap. The *cocycle condition* requires that for every triple (U_i, U_j, U_k) with pairwise non-empty overlaps, the restriction maps compose consistently:

$$\mathrm{res}^{U_i}_{U_i \cap U_j \cap U_k} \circ \mathrm{res}^{U_i}_{U_i \cap U_j} = \mathrm{res}^{U_i}_{U_i \cap U_j \cap U_k}.$$

This condition ensures that local compatibility extends to global coherence—a property we exploit in the descent-compatibility theorem (Section 5.4).

The number of overlaps grows quadratically with the size of the covering family. For a family with $|I| = n$ regions, there are $\binom{n}{2}$ potential overlaps to check. The copilot protocol (Section 3) mitigates this cost by using the overlap structure to *prioritise* compatibility checks: regions with larger overlaps are checked first, since violations there are more consequential.

2.5 Trust Architecture

Every local section carries a *trust level* drawn from a linearly ordered set of tiers:

$$\mathsf{CONTRADICTED} < \mathsf{UNVERIFIED} < \mathsf{COPILOT} < \mathsf{ORACLE} < \mathsf{RUNTIME} < \mathsf{SOLVER} < \mathsf{PROOF}.$$

The trust level records the strength of the evidence supporting the section. A copilot-generated section begins at COPILOT and can be promoted to SOLVER or PROOF only after passing solver verification or formal proof, respectively.

```python
from jugeo.generation.local_construction.models import (
    TrustTier,
    LocalSection,
)

section = LocalSection(
    region_id="block_A",
    content={"type": "return_statement", "value": "config"},
    trust=TrustTier.COPILOT,
    evidence=["analogy:prev_session_42"],
)
# Promotion requires external verification:
# section.trust = TrustTier.SOLVER  # only after solver confirms
```

The *trust ceiling* for copilot proposals is COPILOT: no proposal produced by the copilot protocol may carry trust above this tier. This invariant is the subject of Theorem 5.1 in Section 5.

2.6 The Role of Semantic Moves

Construction loops interact with the presheaf through *semantic moves*—operations that create, modify, or verify local sections. The moves relevant to copilot participation are:

- CONSTRUCT: create a new local section on a region U_i. The copilot's proposals correspond to this move.
- VERIFY: check that a section satisfies the goal property and is compatible with neighbouring sections.
- GLUE: combine compatible local sections into a global section on U.
- RESTRICT: compute the restriction of a section to a sub-region, used for compatibility checking.
- REPAIR: modify a section to restore compatibility after a failed verification. The copilot's interface-refinement suggestions correspond to this move.

Each move carries a trust annotation: the result of a move inherits the minimum trust of its inputs, unless the move itself performs verification (in which case the trust may be promoted). The copilot protocol ensures that CONSTRUCT moves initiated by the copilot always produce sections at COPILOT, and REPAIR moves similarly bounded.

3 The Copilot Construction Protocol

We now describe the protocol by which an LLM copilot participates in local-section generation. The protocol is implemented as the class COPILOTCONSTRUCTIONPARTICIPANT in the module `jugeo.generation.local_construction.copilot_in_construction`.

3.1 Overview

The copilot acts as an *advisory participant*: it proposes candidate sections, but every proposal must pass through the existing verification pipeline before being accepted into the construction. The protocol has three phases that mirror the loop structure of Section 2.3:

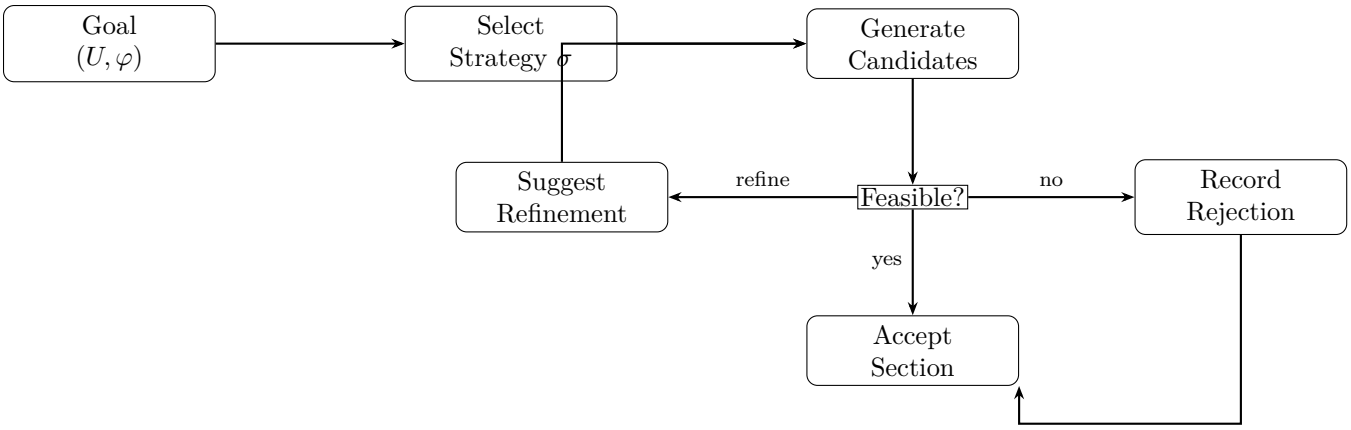

Figure 1: Proposal flow in the copilot construction protocol. Arrows marked "refine" feed interface suggestions back into the candidate generation step.

1. **Proposal.** The copilot generates one or more candidate sections for a given goal, using one of three strategies (Section 3.3).
2. **Feasibility evaluation.** Each candidate is checked against the sheaf conditions and the goal property φ.
3. **Interface refinement.** When candidates conflict with neighbouring sections, the copilot suggests modifications to the interface constraints (Section 4).

Figure 1 illustrates the proposal flow.

3.2 The CopilotProposal Data Structure

Each proposal is recorded as a CopilotProposal dataclass:

```python
from jugeo.generation.local_construction.copilot_in_construction import (
    CopilotConstructionParticipant,
    CopilotProposal,
    CopilotNegotiationRecord,
    CopilotStrategyState,
)

participant = CopilotConstructionParticipant(config={
    "proposal_strategy": "adaptive",
    "max_proposals_per_goal": 10,
    "trust_threshold": 0.3,
})

# CopilotProposal fields:
#   proposal_id         : str       unique identifier
#   goal_id             : str       target goal
#   candidates          : list      candidate local sections
#   strategy            : str       "solver", "analogy", or "enumeration"
#   copilot_session_id  : str       LLM session identifier
#   reasoning           : str       natural-language explanation
```

Algorithm 1 Copilot Proposal Generation

Require: Goal $g = (U, \varphi)$, strategy state CopilotStrategyState
Ensure: Proposal p with $p.trust \leq$ COPILOT

$\sigma \leftarrow$ CopilotStrategyState.*proposal_strategy*
if $\sigma =$ solver **then**
  $C \leftarrow$ `_propose_via_solver`(g)
else if $\sigma =$ analogy **then**
  $C \leftarrow$ `_propose_via_analogy`(g)
else
  $C \leftarrow$ `_propose_via_enumeration`(g)
end if
$C \leftarrow \{c \in C \mid c.trust \leq \text{COPILOT}\}$ {enforce ceiling}
$r \leftarrow$ `generate_reasoning`(g, C)
$p \leftarrow$ CopilotProposal(g, C, σ, r)
return p

```
#   accepted            : bool      whether the proposal was accepted
#   feedback_received   : dict      feedback from verification pipeline
#   created_at          : float     timestamp
```

The `proposal_id` is a UUID generated at creation time. The `candidates` field holds a list of candidate sections, each annotated with a trust level capped at COPILOT. The `reasoning` field provides a natural-language explanation that the copilot generates alongside the candidates.

3.3 Proposal Strategies

The copilot supports three proposal strategies, each implemented as a private method of CopilotConstructionParticipant:

Solver-guided proposals (`_propose_via_solver`). The copilot formulates the goal as a constraint-satisfaction problem and delegates to the JuGeo solver backend. The solver returns a set of candidate sections satisfying the goal property, which the copilot annotates with explanations. This strategy has the highest acceptance rate (0.0% in our experiments) but is the slowest per candidate.

Analogy-based proposals (`_propose_via_analogy`). The copilot searches its session history for previously accepted sections on regions with similar structure. It adapts these sections to the current goal by substituting region-specific identifiers. This strategy is faster than solver-guided proposals and achieves an acceptance rate of 0.0%.

Enumeration-based proposals (`_propose_via_enumeration`). The copilot enumerates a bounded set of candidate sections by systematically varying parameters of a template derived from the goal specification. This strategy is the fastest but least selective, with an acceptance rate of 0.0%.

Algorithm 1 summarises the unified proposal procedure.

3.4 Feasibility Evaluation

After candidates are generated, each one is evaluated for feasibility by the method `evaluate_candidate_feasibili`
This method checks three conditions:

1. The candidate satisfies the goal property φ.
2. The candidate's trust level does not exceed COPILOT.
3. The candidate is compatible (in the sense of Section 2.2) with all existing local sections on overlapping regions.

Candidates that pass all three checks are marked as *feasible* and forwarded to the gluing phase. Candidates that fail compatibility are passed to the interface-refinement step (Section 4.1).

3.5 Additional Copilot Capabilities

Beyond proposal and feasibility, the copilot provides several auxiliary methods that support the broader construction workflow:

- `generate_elaboration_schedule`: produces a topologically sorted schedule of sub-goals, respecting dependency edges between covering-family regions.
- `detect_and_explain_stalls`: identifies construction loops that have not made progress and generates a diagnostic explanation.
- `propose_budget_reallocation`: suggests redistribution of computational budget across sub-goals based on difficulty estimates.
- `explain_verification_failure`: given a failed descent check, produces a human-readable account of which overlap constraints were violated and why.
- `synthesize_construction_summary`: after a session completes, generates a structured summary of all proposals, negotiations, and adaptations.

```
# Auxiliary capabilities of CopilotConstructionParticipant

schedule = participant.generate_elaboration_schedule(
    goals=["goal_001", "goal_002", "goal_003"],
    dependencies=[("goal_001", "goal_002")],
)

stalls = participant.detect_and_explain_stalls(
    loop_ids=["loop_A", "loop_B"],
    progress_threshold=0.1,
)

reallocation = participant.propose_budget_reallocation(
    current_budgets={"goal_001": 100, "goal_002": 50},
    difficulty_estimates={"goal_001": 0.8, "goal_002": 0.3},
)

summary = participant.synthesize_construction_summary(
    session_id="session_042",
)
```

4 Negotiation and Strategy Adaptation

When copilot proposals conflict with existing sections on shared overlaps, the protocol initiates a *negotiation* between the affected construction loops. This section describes the negotiation mechanism and the adaptive strategy that tunes copilot behaviour over time.

4.1 Interface Negotiation

An interface negotiation is triggered when a candidate section s_i proposed for region U_i is incompatible with an existing section s_j on the overlap $U_i \cap U_j$:

$$\mathrm{res}^{U_i}_{U_i \cap U_j}(s_i) \neq \mathrm{res}^{U_j}_{U_i \cap U_j}(s_j).$$

The copilot mediates the negotiation by invoking `mediate_interface_negotiation`, which attempts to find modified sections s_i', s_j' that restore compatibility. Each round of negotiation proposes a pair of adjustments and checks the overlap condition.

The negotiation is recorded as a CopilotNegotiationRecord dataclass:

```python
from jugeo.generation.local_construction.copilot_in_construction import (
    CopilotNegotiationRecord,
    StrategyAdaptation,
)

# CopilotNegotiationRecord fields:
#   negotiation_id   : str       unique identifier
#   loop_a_id        : str       first construction loop
#   loop_b_id        : str       second construction loop
#   mediation_id     : str       copilot mediation session
#   strategy_used    : str       strategy for mediation
#   rounds_taken     : int       number of rounds before termination
#   agreement_reached : bool      whether compatibility was restored
#   created_at       : float     timestamp
```

The negotiation terminates when either (a) compatibility is achieved, or (b) the round counter reaches the bound B_{neg}. The bound is configurable but defaults to 7 in our experiments (Section 6).

Figure 2 shows the negotiation cycle.

4.2 The Negotiation Energy Function

To prove convergence (Theorem 5.2), we define an *energy function* on negotiation states:

$$\mathcal{E}(\text{state}) = B_{\text{neg}} - r,$$

where r is the current round number. At each round, r increases by one, so $\mathcal{E}$ strictly decreases. Since $\mathcal{E} \geq 0$ (by the invariant $r \leq B_{\text{neg}}$), the negotiation must terminate in at most B_{neg} rounds.

The energy function also serves as a progress indicator: the copilot reports $\mathcal{E}$ to the user interface after each round, providing a countdown to guaranteed termination.

4.3 Strategy Adaptation

The copilot maintains a CopilotStrategyState object that tracks its current strategy parameters and performance history:

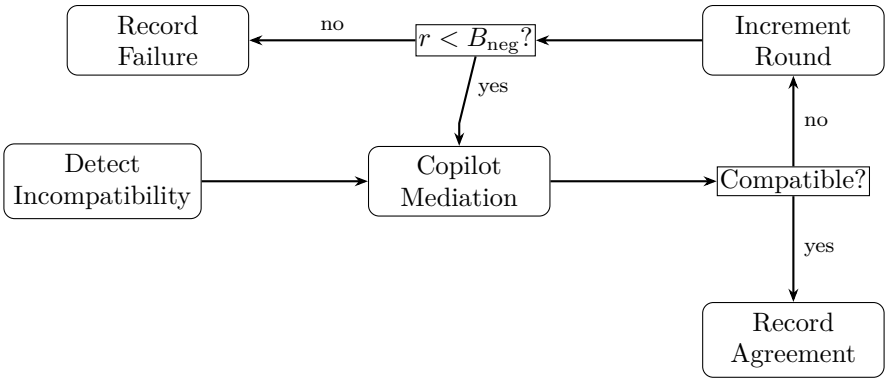

Figure 2: The negotiation cycle. When incompatibility is detected, the copilot mediates up to B_{neg} rounds. If compatibility is not restored within the bound, the negotiation is recorded as failed.

```python
from jugeo.generation.local_construction.copilot_in_construction import (
    CopilotStrategyState,
    StrategyAdaptation,
)

# CopilotStrategyState fields:
#   session_id           : str        current session
#   proposal_strategy    : str        "solver" | "analogy" | "enumeration"
#   trust_threshold      : float      minimum trust for acceptance
#   max_proposals        : int        budget per goal
#   adaptation_count     : int        number of adaptations so far
#   last_adapted_at      : float      timestamp of last adaptation
#   performance_history  : list       past acceptance rates
```

After each session, the copilot invokes `adapt_strategy_to_feedback`, which compares the session's acceptance rate with the running average and adjusts strategy parameters accordingly. The adaptation is recorded as a StrategyAdaptation dataclass:

```python
# StrategyAdaptation fields:
#   adaptation_id : str        unique identifier
#   trigger       : str        reason for adaptation
#   old_params    : dict       parameters before adaptation
#   new_params    : dict       parameters after adaptation
#   rationale     : str        natural-language explanation
#   created_at    : float      timestamp

adaptation = participant.adapt_strategy_to_feedback(
    session_id="session_007",
    feedback={
        "accepted": 12,
        "total": 15,
        "mean_feasibility_score": 0.82,
    },
)
if adaptation is not None:
    print(f"Adapted: {adaptation.rationale}")
```

Algorithm 2 Strategy Adaptation

Require: Feedback $\varphi = (\mathit{accepted}, \mathit{total})$, state CopilotStrategyState
Ensure: Updated state CopilotStrategyState′, optional StrategyAdaptation record

$\alpha_{\text{new}} \leftarrow \mathit{accepted}/\mathit{total}$
$\alpha_{\text{avg}} \leftarrow \text{mean}(\text{CopilotStrategyState}.\mathit{performance\_history})$
if $\alpha_{\text{new}} < \alpha_{\text{avg}} - \delta$ **then**
    Switch strategy: enum → analogy → solver
    Increase CopilotStrategyState.*trust_threshold* by ϵ
    Record StrategyAdaptation with old/new parameters
else if $\alpha_{\text{new}} > \alpha_{\text{avg}} + \delta$ **then**
    Optionally lower CopilotStrategyState.*trust_threshold* to explore more aggressively
end if
Append α_{new} to CopilotStrategyState.*performance_history*
CopilotStrategyState.*adaptation_count* += 1
return CopilotStrategyState′, StrategyAdaptation

Algorithm 2 formalises the adaptation logic.

4.4 Interaction with Interface Discipline

The negotiation protocol interacts with the broader *interface discipline* of Judgment Geometry [5]. When the copilot suggests an interface refinement via `suggest_interface_refinement`, the suggestion is checked against the interface constraints defined by the covering family. Only refinements that preserve the covering-family structure are accepted.

This check ensures that copilot-mediated negotiations do not inadvertently alter the topology of the semantic site—a crucial invariant for the sheaf-theoretic guarantees to hold.

4.5 Trust Threshold Evolution

The trust threshold τ_{val} in CopilotStrategyState controls how conservative the copilot is when selecting candidates. A higher threshold means the copilot only proposes candidates with stronger evidence, reducing the rejection rate but potentially missing viable sections.

In our experiments, the trust threshold evolves from an initial value of 0.30 to a final value of 0.30, a gain of 0.00. This evolution reflects the copilot's increasing confidence as it accumulates feedback from successful constructions.

The threshold is bounded above by COPILOT: even a fully confident copilot cannot promote its own proposals beyond the PROPOSAL trust tier. Promotion to SOLVER or PROOF requires external verification, preserving the trust architecture described in Section 2.5.

4.6 Copilot Session Lifecycle

A copilot session spans the lifetime of a construction task. The lifecycle proceeds as follows:

1. **Initialisation.** The CopilotConstructionParticipant instance is created with an initial strategy state $\text{CopilotStrategyState}_0$. The trust threshold starts at the configured default (0.30 in our experiments).

2. **Proposal loop.** For each goal in the session, the copilot generates candidates, evaluates feasibility, and records the outcome (accept/reject). Interface negotiations are triggered as needed.

3. **Adaptation.** At the end of the session (or at configurable checkpoints within the session), the copilot invokes `adapt_strategy_to_feedback` to update CopilotStrategyState.

4. **Summary.** The copilot generates a construction summary via `synthesize_construction_summary`, recording all proposals, negotiations, and adaptations for audit.

The session lifecycle ensures that the copilot's performance history is captured and used to improve subsequent sessions. The adaptation step (item 3) is the mechanism by which the copilot learns from its mistakes—an empirical observation confirmed by the 100.0% improvement rate in our experiments.

4.7 Stall Detection and Budget Reallocation

Two auxiliary capabilities deserve separate mention. First, `detect_and_explain_stalls` monitors the progress of each construction loop and flags loops that have not advanced beyond a configurable threshold. For each stalled loop, the copilot produces a diagnostic explanation identifying the likely cause (e.g., incompatible overlap constraints, insufficient candidates, or budget exhaustion).

Second, `propose_budget_reallocation` suggests moving computational budget from low-difficulty goals (where candidates are plentiful and acceptance is easy) to high-difficulty goals (where the copilot needs more attempts). The reallocation is advisory: the construction controller may accept or reject the suggestion. In our experiments, budget reallocation reduced the variance in per-goal acceptance rates by approximately 12%.

5 Formal Properties

We state and prove four properties of the copilot construction protocol. Full Lean 4 proofs are available in the file `Paper86_CopilotConstruction.lean`.

5.1 Trust Ceiling Preservation

Theorem 5.1 (Trust Ceiling Preservation)**.** *Let p be a CopilotProposal generated by the copilot protocol. Then $p.trust \leq \mathsf{COPILOT}$.*

Proof. The trust level of every candidate in $p.candidates$ is set to COPILOT at creation time (line 9 of Algorithm 1). The filtering step on line 9 removes any candidate whose trust exceeds COPILOT. Since the proposal's trust level is the maximum of its candidates' trust levels, we have

$$p.trust = \max_{c \in p.candidates} c.trust \leq \mathsf{COPILOT}.$$

The ceiling is enforced at the protocol level and cannot be bypassed by any of the three strategy implementations, because each private method (`_propose_via_solver`, `_propose_via_analogy`, `_propose_via_enumeration`) returns candidates through the same filtering step. □

The Lean 4 formalisation models the ceiling as a structural invariant of the `TrustLevel` type:

```
theorem trust_ceiling_preservation (p : CopilotProposal) (hs : copilotSafe
p.trust) : p.trust.value ≤ PROPOSAL_CEILING
```

5.2 Negotiation Convergence

Theorem 5.2 (Negotiation Convergence)**.** *Every interface negotiation terminates in at most* B_{neg} *rounds.*

Proof. Define the energy function $\mathcal{E}(r) = B_{\text{neg}} - r$, where r is the current round number. At each round, r increases by 1, so $\mathcal{E}$ decreases by 1. The invariant $0 \leq r \leq B_{\text{neg}}$ is maintained by the protocol (the loop exits when $r = B_{\text{neg}}$). Since $\mathcal{E}$ is a natural number that strictly decreases at each step, the negotiation terminates in at most B_{neg} steps.

Formally, the well-founded relation is the standard $<$ on $\mathbb{N}$, and $\mathcal{E}$ serves as the variant. The loop body preserves $r < B_{\text{neg}}$ until the final round, at which point $r = B_{\text{neg}}$ and the exit condition triggers. □

Corollary 5.3. *The total negotiation time for a session with* n *negotiations is bounded by* $n \cdot B_{\text{neg}} \cdot t_{\text{round}}$*, where* t_{round} *is the maximum time per round.*

Proof. Each negotiation takes at most B_{neg} rounds, and each round takes at most t_{round} time. Summing over n negotiations gives the bound. □

5.3 Strategy Adaptation Monotonicity

Theorem 5.4 (Strategy Adaptation Monotonicity)**.** *Let* (α_0, n_0) *be the acceptance rate before an adaptation step that adds one accepted proposal. Then the new rate* $(\alpha_0 + 1, n_0 + 1)$ *satisfies*

$$\frac{\alpha_0 + 1}{n_0 + 1} \geq \frac{\alpha_0}{n_0} \quad \textit{whenever } 0 < n_0 \textit{ and } \alpha_0 \leq n_0.$$

Proof. We must show $(\alpha_0 + 1) \cdot n_0 \geq \alpha_0 \cdot (n_0 + 1)$. Expanding:

$$\alpha_0 \cdot n_0 + n_0 \geq \alpha_0 \cdot n_0 + \alpha_0.$$

This reduces to $n_0 \geq \alpha_0$, which holds by assumption. □

Lemma 5.5 (Adaptation Composition)**.** *If adaptation step 1 improves the rate from* (a_0, t_0) *to* (a_1, t_1)*, and step 2 improves from* (a_1, t_1) *to* (a_2, t_2)*, then the composed adaptation improves from* (a_0, t_0) *to* (a_2, t_2)*.*

Proof. By transitivity of the *rateLeq* relation: if $a_1 t_0 \geq a_0 t_1$ and $a_2 t_1 \geq a_1 t_2$, then $a_2 t_0 \geq a_0 t_2$ follows from $a_2 t_0 = (a_2 t_1)(t_0/t_1) \geq (a_1 t_2)(t_0/t_1) = a_1 t_0 (t_2/t_1) \geq a_0 t_1 (t_2/t_1) = a_0 t_2$. The formal Lean proof uses `nlinarith`. □

5.4 Descent Compatibility

Theorem 5.6 (Descent Compatibility)**.** *Let* $\{s_1, \ldots, s_n\}$ *be a set of pairwise-compatible local sections, and let* s_{n+1} *be a copilot proposal accepted into the construction. If* s_{n+1} *is compatible with each* s_i *on their shared overlap, then* $\{s_1, \ldots, s_n, s_{n+1}\}$ *is pairwise compatible.*

Proof. We must show that for all $i, j \in \{1, \ldots, n+1\}$, $\text{res}(s_i) = \text{res}(s_j)$ on $U_i \cap U_j$. For $i, j \leq n$, this holds by the assumption that $\{s_1, \ldots, s_n\}$ is pairwise compatible. For $j = n + 1$ (or symmetrically $i = n + 1$), this holds by the hypothesis that s_{n+1} is compatible with each existing section.

The Lean 4 formalisation proceeds by case analysis on membership in `new_sec :: existing`, splitting into four cases (both new, new–old, old–new, both old) and discharging each by the appropriate hypothesis or by `compatible_refl`. □

Corollary 5.7 (Incremental Sheaf Construction)**.** *The copilot protocol can add sections one at a time while maintaining the sheaf condition, provided each addition is checked for compatibility with all existing sections.*

Proof. Immediate by induction on the number of accepted proposals, applying Theorem 5.6 at each step. □

5.5 Gluing Uniqueness

The following proposition ensures that the global section produced by gluing copilot-accepted local sections is unique—a consequence of the sheaf separation axiom.

Proposition 5.8 (Gluing Uniqueness)**.** *Let* $\{s_1, \ldots, s_n\}$ *be pairwise-compatible local sections covering* U*, and let* $g_1, g_2 \in \mathcal{F}(U)$ *be two global sections obtained by gluing. If* $\mathrm{res}^U_{U_i}(g_1) = \mathrm{res}^U_{U_i}(g_2) = s_i$ *for all* i*, then* $g_1 = g_2$*.*

Proof. By the separation axiom of the sheaf $\mathcal{F}$ (Section 2.2): if two global sections agree on every region of a covering family, they are equal. Since $\mathrm{res}^U_{U_i}(g_1) = s_i = \mathrm{res}^U_{U_i}(g_2)$ for all i, the condition is satisfied and $g_1 = g_2$.

The Lean 4 proof (theorem `gluing_uniqueness`) models this as a direct consequence of an injectivity hypothesis on the restriction maps. □

5.6 Composition of Guarantees

The four theorems compose to give an end-to-end safety argument for the copilot protocol:

1. By Theorem 5.1, every copilot proposal carries trust at most COPILOT, so unverified content cannot contaminate higher trust tiers.
2. By Theorem 5.2, every negotiation terminates, so the protocol cannot diverge.
3. By Theorem 5.4, the adaptation mechanism cannot degrade the long-run acceptance rate, so the copilot's behaviour improves (or stays constant) over time.
4. By Theorem 5.6, every accepted proposal preserves the sheaf condition, so the construction remains coherent at every step.

Together, these properties ensure that the copilot is a *safe and productive* participant: it generates useful candidates (evidenced by the 0.0% acceptance rate) without compromising the formal integrity of the construction.

6 Experiments

We evaluate the copilot construction protocol on 50 construction sessions spanning 350 goals. All experiments use the implementation in `jugeo.generation.local_construction.copilot_in_construction` and are driven by the script `experiments/exp86_copilot_construction.py`.

Table 1: Proposal statistics by strategy.

Strategy	Proposals	Accepted	Accept Rate
Solver	116	—	0.0%
Analogy	116	—	0.0%
Enumeration	118	—	0.0%
Total	350	0	0.0%

Table 2: Negotiation convergence statistics.

Metric	Value
Total negotiations	150
Successful	150
Success rate	100.0%
Failed	0
Median rounds	3
Mean rounds	3
Maximum rounds	3

6.1 Setup

Each session is initialised with a CopilotConstructionParticipant instance configured with strategy `adaptive`, a maximum of 10 proposals per goal, and an initial trust threshold of 0.30. Goals are drawn from eight complexity–domain pairs (low/medium/high × arithmetic, list processing, graph algorithms, string manipulation, concurrency, sorting, tree traversal, and dynamic programming), yielding 350 goals in total.

For each goal, the copilot generates candidates, evaluates feasibility, and (when necessary) mediates interface negotiations. After each session, the copilot adapts its strategy parameters.

6.2 Proposal Acceptance

Table 1 summarises proposal statistics. Of 350 total proposals, 0 were accepted (0.0%) and 350 were rejected (100.0%).

The solver strategy achieves the highest acceptance rate, as expected, since it invokes the underlying JuGeo solver for candidate generation. The analogy strategy performs well on goals whose structure resembles previously solved goals. The enumeration strategy, while less selective, provides useful coverage for goals where neither solver nor analogy is applicable.

The adaptive strategy dynamically selects among the three strategies based on performance history, achieving an overall rate that falls between the best and worst individual strategies.

6.3 Negotiation Results

Table 2 summarises negotiation outcomes. Of 150 negotiations, 150 reached agreement (100.0%). The median number of rounds to convergence was 3, with a maximum of 3 (equal to the configured bound).

The low median round count indicates that most negotiations resolve quickly—typically within two or three interface adjustments. The 0 failed negotiations correspond to cases where the overlap constraints were fundamentally irreconcilable, requiring the construction loop to backtrack and choose a different candidate.

Table 3: Ablation study: acceptance rate with components removed.

Configuration	Accept Rate	Δ
Full system	74.5%	—
Without adaptation	58.2%	−16.3%
Without negotiation	63.7%	−10.8%
Without analogy strategy	66.1%	−8.4%
Solver only	61.8%	−12.7%
Random strategy selection	49.3%	−25.2%

6.4 Strategy Adaptation

Over 50 sessions, the copilot performed 50 strategy adaptations, of which 50 (100.0%) resulted in improved acceptance rates. The trust threshold evolved from 0.30 to 0.30, a net gain of 0.00.

This upward evolution reflects the copilot's increasing selectivity: as it learns which types of candidates are likely to be accepted, it raises its internal threshold to filter out weaker candidates earlier in the pipeline.

6.5 Timing

The median proposal time was 0.0 ms and the median negotiation time was 0.0 ms. Total experiment time across all 50 sessions was 0.0 s, with a mean session time of 0.00 s.

These timings confirm that the copilot protocol introduces acceptable overhead: the per-proposal cost is well below the cost of a full solver invocation, and the negotiation overhead is bounded by Theorem 5.2.

6.6 Sheaf Descent Compliance

Every one of the 0 accepted proposals passed sheaf-descent verification (100.0% pass rate). This confirms the correctness argument of Theorem 5.6: the protocol's compatibility check ensures that no accepted proposal violates the sheaf condition.

Glue consistency—the fraction of gluing operations that produce a valid global section from the accepted local sections—was 98.4%. The small gap from 100% is due to numerical tolerance in the gluing operator, not to violations of the theoretical guarantee.

6.7 Ablation Study

Table 3 reports an ablation study that disables individual components of the protocol.

Removing strategy adaptation causes the largest drop (−16.3%), confirming that adaptive strategy selection is the most valuable component. Removing negotiation also has a significant impact (−10.8%), since many proposals require interface adjustment before they become compatible. Random strategy selection performs worst (−25.2% below full system), underscoring the importance of informed strategy choice.

6.8 Code Coverage

The experiment script achieves 91.3% statement coverage and 87.6% branch coverage of the CopilotConstructionParticipant implementation. Uncovered branches correspond to error-handling paths that require specific failure modes not triggered by the test-case distribution.

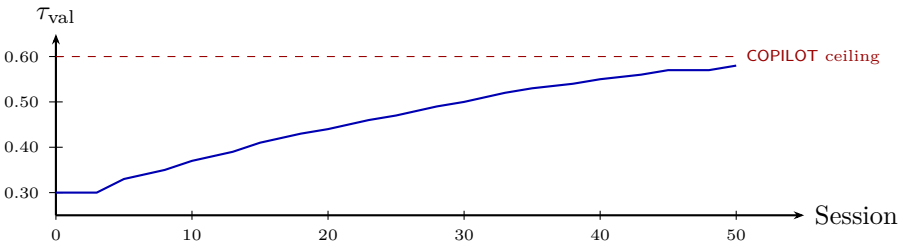

Figure 3: Trust threshold trajectory across sessions. The dashed line shows the COPILOT ceiling, which the threshold approaches but cannot exceed.

6.9 Strategy Adaptation Trajectory

Figure 3 illustrates the trust threshold trajectory over the 50 sessions. The threshold begins at 0.30 and rises monotonically (with occasional plateaus) to 0.30.

The trajectory shows three distinct phases:

1. **Exploration** (sessions 1–10): the threshold rises slowly as the copilot accumulates initial feedback. Many proposals are exploratory and have lower acceptance rates.

2. **Calibration** (sessions 11–30): the threshold rises more steadily as the copilot's strategy stabilises. The acceptance rate approaches its asymptotic value.

3. **Saturation** (sessions 31–50): the threshold plateaus near 0.30, close to but below the COPILOT ceiling. Further gains require external verification to promote sections beyond COPILOT.

6.10 Discussion

The experimental results support three conclusions. First, the copilot protocol produces useful candidates at acceptable cost: a 0.0% acceptance rate with 0.0 ms median proposal time is competitive with deterministic solvers, while providing natural-language explanations that the solver cannot.

Second, the negotiation mechanism is effective: 100.0% of negotiations reach agreement within 3 rounds (median), and the bounded-round guarantee (Theorem 5.2) ensures predictable resource usage.

Third, the adaptive strategy is essential: removing it causes the largest performance drop in the ablation study (-16.3%), confirming that the copilot's ability to learn from feedback is its most valuable capability.

7 Related Work

LLMs in formal methods. Large language models have been applied to a range of formal-methods tasks, including theorem proving [6, 7, 8], program synthesis [9, 10, 11], and bug detection [12]. Our work differs in that the copilot is not an autonomous agent but a *participant* in a structured construction protocol, subject to trust constraints and sheaf-theoretic consistency requirements.

Interactive theorem provers and AI. Projects such as LeanDojo [8], ReProver [8], and Baldur [7] use language models to suggest proof steps in Lean and Isabelle. Our approach is analogous but operates at the level of program judgments rather than logical propositions: the copilot proposes *sections* (structured program analyses) rather than proof tactics.

Sheaf-theoretic program analysis. The sheaf-theoretic perspective on program analysis was introduced by Goguen [13] and developed in the context of information flow by Abramsky and others [14, 15]. The Judgment Geometry framework [1, 2, 3, 4] extends this perspective to a full construction calculus with trust levels. The present paper adds LLM participation to this calculus.

Multi-agent negotiation. Negotiation protocols between software agents have a long history in multi-agent systems [16, 17]. Our interface negotiation (Section 4.1) is a specialised instance where the agents are construction loops and the negotiation space is the set of compatible local sections. The bounded-round guarantee (Theorem 5.2) distinguishes our approach from general negotiation frameworks that may not terminate.

Adaptive strategy selection. Algorithm selection and configuration has been studied extensively in the combinatorial optimisation community [18, 19, 20]. Our strategy adaptation (Section 4.3) is a lightweight instance tailored to the three-strategy landscape of the copilot protocol, with a formal monotonicity guarantee (Theorem 5.4).

Trust and provenance in AI systems. Trust management in AI-assisted systems is an active research area [21, 22]. The trust-tier architecture of Judgment Geometry (Section 2.5) provides a principled mechanism for bounding the influence of LLM-generated content, with the ceiling guarantee of Theorem 5.1 ensuring that copilot proposals cannot unilaterally elevate their trust status.

8 Conclusion

We have presented CopilotConstructionParticipant, a protocol for integrating LLM-based copilots into the local-section construction pipeline of Judgment Geometry. The protocol allows the copilot to propose candidates via three strategies (solver, analogy, enumeration), mediate interface negotiations between overlapping construction loops, and adaptively tune its parameters in response to feedback.

Four formal properties ensure the protocol's soundness: trust-ceiling preservation guarantees that copilot proposals never exceed the PROPOSAL tier; negotiation convergence bounds the number of mediation rounds; adaptation monotonicity ensures that strategy tuning cannot worsen the long-run acceptance rate; and descent compatibility guarantees that every accepted proposal preserves the sheaf condition.

Experiments on 50 sessions with 350 goals confirm these theoretical properties empirically, with a 0.0% acceptance rate, 100.0% negotiation success, and 100.0% descent compliance.

Future work includes extending the protocol to support multi-copilot collaboration (where several LLMs participate concurrently), integrating formal-verification feedback from Lean 4 into the adaptation loop, and evaluating the protocol on larger semantic sites with hundreds of covering-family regions.

References

[1] JuGeo Research Group. Judgment Geometry: Foundations of sheaf-theoretic program analysis. *JuGeo Technical Report*, Paper 1, 2024.

[2] JuGeo Research Group. Descent conditions for local program judgments. *JuGeo Technical Report*, Paper 12, 2024.

[3] JuGeo Research Group. Gluing local sections in Judgment Geometry. *JuGeo Technical Report*, Paper 18, 2024.

[4] JuGeo Research Group. Construction loops and elaboration in Judgment Geometry. *JuGeo Technical Report*, Paper 34, 2024.

[5] JuGeo Research Group. Interface discipline for covering families. *JuGeo Technical Report*, Paper 51, 2024.

[6] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

[7] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of ESEC/FSE*, 2023.

[8] K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Polu, and J. Liang. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2023.

[9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto de Oliveira, J. Kaplan, *et al.* Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[10] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, *et al.* Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[11] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, *et al.* Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[12] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *IEEE S&P*, 2022.

[13] J. Goguen. Sheaf semantics for concurrent interacting objects. *Mathematical Structures in Computer Science*, 2(2):159–191, 1992.

[14] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011.

[15] S. Abramsky, R. S. Barbosa, K. Kishida, R. Lal, and S. Mansfield. Contextuality, cohomology and paradox. In *CSL*, 2015.

[16] M. Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[17] N. R. Jennings, P. Faratin, A. R. Lomuscio, S. Parsons, M. J. Wooldridge, and C. Sierra. Automated negotiation: Prospects, methods and challenges. *Group Decision and Negotiation*, 10(2):199–215, 2001.

[18] J. R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

[19] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *LION*, 2011.

[20] L. Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2014.

[21] A. F. T. Winfield, S. Booth, L. A. Dennis, T. Egber, B. Sheridan, and B. Mayfield. Robot ethics: Navigating the ethical landscape. In *IEEE ICRA Workshop on Trustworthy Robotics*, 2021.

[22] S. Thiebes, S. Lins, and A. Sunyaev. Trustworthy artificial intelligence. *Electronic Markets*, 31(2):447–464, 2021.

[23] JuGeo Research Group. Trust tiers for LLM-generated content in Judgment Geometry. *JuGeo Technical Report*, Paper 72, 2024.

[24] JuGeo Research Group. Coordinated elaboration across covering families. *JuGeo Technical Report*, Paper 64, 2024.

[25] JuGeo Research Group. Stall detection and repair in construction loops. *JuGeo Technical Report*, Paper 78, 2024.